On-Board Vision-Based Quadrupedal Navigation using a Single RGB-D Camera and 3D Gaussian Splatting Testbed

Minseok Lee[1], Seongyu Han[1], Heejae Moon[1], and Sung Soo Hwang[1*]

[1]School of Artificial Intelligence, Computer and Electrical Engineering,
Handong Global University, Pohang 37554, Republic of Korea
({glen, tjsrb439, 22000249}@handong.ac.kr, sshwang@handong.edu) * Corresponding author

Abstract: Directly deploying autonomous navigation systems on physical quadrupedal platforms in real-world settings presents severe challenges due to hardware safety hazards, time-inefficient trial-and-error runs, and the high-frequency camera "bobbing noise" (ego-motion jitter) inherent to legged locomotion. This shaking degrades RGB-D visual feature tracking and depth stream consistency. To address these limitations, this paper proposes a practical, bifurcated sim-to-real framework for RGB-D camera-based quadrupedal navigation. In the simulation stage, we reconstruct a high-fidelity 3D Gaussian Splatting (3DGS) digital twin of an indoor corridor and utilize a reinforcement learning (RL) locomotion policy to actively emulate walking dynamics and generate representative camera shaking noise. Under these simulated disturbances, the parameters of the on-board RTAB-Map RGB-D Visual SLAM and Nav2 navigation stack are safely pre-tuned and optimized. In the deployment stage, the computationally intensive RL policy is bypassed, and navigation velocity commands are routed directly to the robot's proprietary Sport API via a lightweight ROS 2 bridge to respect the strict embedded computing limits of the NVIDIA Jetson Orin NX. Experimental results demonstrate that navigation and SLAM parameters optimized under the simulated bobbing noise exhibit robust transferability to the physical Unitree Go2 robot, achieving stable visual mapping and autonomous obstacle avoidance without real-world parameter retuning.

Keywords: Quadrupedal Locomotion, Visual SLAM, Autonomous Navigation, Sim-to-Real Transfer, 3D Gaussian Splatting, Ego-motion Noise.

1. INTRODUCTION

Autonomous quadrupedal robots have attracted substantial attention due to their superior mobility in traversing challenging and unstructured terrains compared to conventional wheeled systems. Achieving fully autonomous operations in indoor environments requires the seamless integration of stable legged locomotion and high-fidelity simultaneous localization and mapping (SLAM).

However, deploying visual navigation using a single RGB-D camera on a legged platform introduces two critical sim-to-real challenges. First is the *perceptual sim-to-real gap*: standard physics simulators generate synthetic environments that lack photo-realistic visual textures and depth fidelity. This discrepancy causes feature-based visual SLAM algorithms to fail during real-world deployment due to severe feature degradation and matching errors. Second is *locomotion-induced ego-motion noise* (commonly referred to as bobbing noise): unlike wheeled platforms, legged locomotion inherently generates periodic, high-frequency camera oscillations. This shaking causes motion blur in the RGB stream—disrupting visual odometry (VO) tracking—and introduces temporal depth estimation discrepancies that manifest as phantom obstacles in local costmaps. Directly tuning navigation parameters to mitigate these dynamics on physical hardware is time-intensive and carries substantial risks of hardware damage from collision-induced failures.

To address these limitations, we propose a practical, bifurcated Sim-to-Real integration framework for quadrupedal visual navigation utilizing a single front-facing RGB-D camera (Intel RealSense D435i) interfaced with an NVIDIA Jetson Orin NX. Our framework decouples virtual-space parameter verification from physical-space system deployment. In the simulation phase, we reconstruct a photorealistic digital twin of an indoor corridor using 3D Gaussian Splatting (3DGS). Because the robot's proprietary, low-level controller cannot be integrated directly into the physics simulator, we train a model-free RL locomotion policy in Isaac Lab to emulate physical gait characteristics, thereby generating representative camera ego-motion noise. Under this synthesized visual and dynamic environment, we optimize the parameters of the RTAB-Map and Nav2 frameworks. Upon physical deployment, the deep neural network policy is bypassed, and navigation commands are routed to the manufacturer's built-in Sport API via a lightweight ROS 2 bridge, reserving the edge computer's resource budget for visual SLAM and navigation.

The contributions of this work are summarized as follows:

1. We establish a photorealistic visual validation sandbox using 3DGS and FVDB reconstruction, addressing the perceptual sim-to-real gap by providing realistic RGB textures and consistent depth characteristics for visual SLAM validation.
2. We model quadruped-specific camera ego-motion (bobbing) noise in simulation via a reinforcement learning locomotion policy, enabling safe and robust pre-tuning of navigation and mapping parameters.
3. We present a resource-efficient deployment architecture that interfaces high-level ROS 2 navigation commands with the robot's native Sport API, achieving suc-

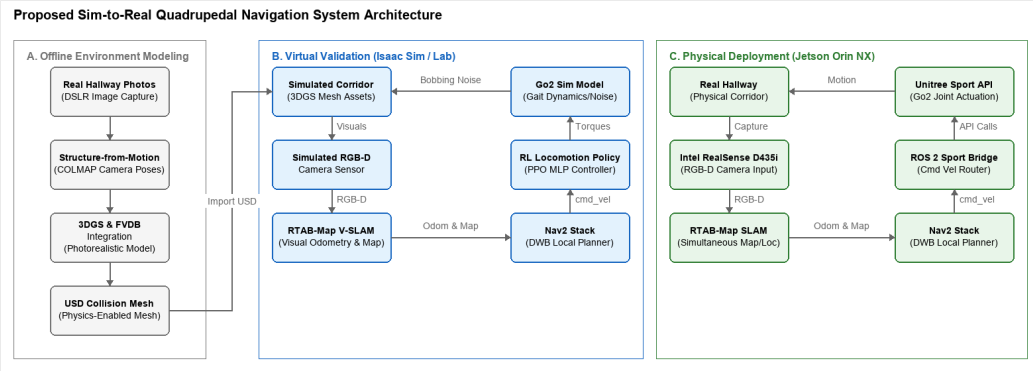

Fig. 1. Overall architecture of the proposed Sim-to-Real quadrupedal navigation framework. (a) In the virtual validation stage, we build a high-fidelity 3DGS environment and train an RL locomotion policy to simulate camera bobbing noise for upper-level navigation tuning. (b) In the physical deployment stage, the RL policy is bypassed, and commands are sent to the Unitree Sport API via a ROS 2 bridge to optimize computational resources on the Jetson Orin NX.

cessful zero-shot transfer of parameters tuned under simulated bobbing noise onto a single Jetson Orin NX.

2. RELATED WORK

2.1 Reinforcement Learning for Quadrupedal Locomotion

Reinforcement learning (RL) has significantly advanced control paradigms for legged systems. Classical model-based methods, such as Model Predictive Control (MPC) and Whole-Body Control (WBC), rely heavily on accurate system dynamics and environmental contact models. Conversely, model-free RL algorithms enable quadrupedal platforms to acquire robust locomotion policies through trial-and-error interactions. The advent of highly parallelized physics simulators, such as NVIDIA Isaac Sim and Isaac Lab, has facilitated the training of deep RL policies across thousands of concurrent environments. This paradigm has enabled successful real-world deployments of agile legged locomotion on systems such as ANYmal and Unitree Go1/Go2. Through domain randomization and curriculum learning, researchers have minimized the dynamics gap, allowing policies to transfer directly from simulation to physical platforms. However, most existing locomotion research focuses primarily on gait stability and disturbance rejection, leaving high-level visual navigation integration and sensor noise characteristics largely unaddressed.

2.2 Visual SLAM and Navigation Integration

Mobile robot navigation has classically relied on Light Detection and Ranging (LiDAR) sensors due to their high spatial accuracy and wide field of view. Nonetheless, LiDARs are often cost-prohibitive, heavy, and power-intensive, making them less suitable for payload- and power-constrained quadruped robots. Visual SLAM (VSLAM) utilizing RGB-D cameras offers a lightweight, cost-effective alternative. Among state-of-the-art visual SLAM frameworks, Real-Time Appearance-Based Mapping (RTAB-Map) is widely adopted due to its ability to perform appearance-based loop closure detection combined with pose-graph optimization, producing drift-minimized trajectory estimates. Simultaneously, the ROS 2 Navigation (Nav2) framework has established itself as the industry standard for path planning and obstacle avoidance.

Although recent works have integrated legged locomotion with visual SLAM, their validation is typically confined to simplistic, synthetic virtual environments. These idealized simulation spaces do not reflect the visual and geometric complexities of actual physical buildings, resulting in domain transfer failures. This work mitigates this discrepancy by reconstructing high-fidelity, photorealistic digital twins via 3DGS, bridging the visual-spatial gap between simulation and reality.

3. SYSTEM AND LOCOMOTION CONTROL

3.1 System Architecture

The proposed framework integrates RTAB-Map Visual SLAM, the Nav2 navigation stack, and a bifurcated locomotion control architecture. The control architecture utilizes a Deep Reinforcement Learning (DRL) locomotion policy for virtual validation and transitions to the robot's native, low-level Sport API for physical deployment. In the simulation phase, the DRL policy—trained via Proximal Policy Optimization (PPO) in Isaac Lab (v2.3.2) on Isaac Sim (v5.1.0)—is employed to generate representative quadrupedal gait dynamics and camera ego-motion (bobbing) noise. High-level velocity commands ($\mathbf{v_{cmd}} = [\mathbf{v_x}, \mathbf{v_y}, \omega_\mathbf{z}]^\mathbf{T}$) computed by the Nav2 local planner are mapped directly to the policy's action space. During real-world deployment, the computationally heavy DRL policy is bypassed to conserve edge computational resources, and Nav2 velocity commands are routed directly to the Unitree Sport API via a custom ROS 2 bridge. This resource-efficient architecture optimizes the computational budget of the Jetson Orin NX. The system architecture is illustrated in Fig. 1.

3.2 Locomotion Learning in Isaac Sim

To synthesize representative legged locomotion dynamics and emulate physical camera bobbing noise in simulation, we train an RL policy in NVIDIA Isaac Lab (v2.3.2), built upon the Isaac Sim (v5.1.0) physics engine. The policy functions as a dynamic perturbation generator, producing walking-induced camera oscillations. We utilize the Procedural Terrain Generator in Isaac Lab to construct a training environment with mixed terrains, including uneven ground, ramps, and step obstacles. We employ curriculum learning to dynamically scale the terrain difficulty (e.g., obstacle heights and slope angles) in proportion to the robot's traversal success rate. To enhance the robustness of the policy and bridge the sim-to-real dynamics gap, we apply the following domain randomizations:

- Torso mass randomization: $\Delta m \in [-1.0, +3.0]$ kg.
- Ground friction coefficient: $\mu \in [0.6, 0.8]$.
- External disturbances: impulse force perturbations applied to the robot's center of mass (CoM).

The physics simulation step size is configured to 200 Hz, and the policy control loop executes at 50 Hz (decimation factor of 4).

The policy is trained using the PPO implementation in the PyTorch-based `skrl` library. Both the actor and critic networks are parameterized as Multilayer Perceptrons (MLPs) with hidden layers of $[512, 256, 128]$ and Exponential Linear Unit (ELU) activation functions. The initial learning rate is 1.0×10^{-3}, regulated by a KL-divergence adaptive scheduler with a target threshold of 0.01. The discount factor is $\gamma = 0.99$, and the Generalized Advantage Estimator (GAE) parameter is $\lambda = 0.95$.

The reward function is formulated to track velocity references while maintaining stable legged posture. The total reward R is defined as:

$$R = w_{lin}R_{lin} + w_{ang}R_{ang} + w_{air}R_{air} - \sum_i w_{p,i}P_i \quad (1)$$

where:

- $R_{lin} = \exp(-\|\mathbf{v}^*_{\mathbf{xy}} - \mathbf{v}_{\mathbf{xy}}\|^2_2/\sigma^2_{\mathbf{lin}})$ is the linear velocity tracking reward, where $\mathbf{v}^*_{\mathbf{xy}}$ and $\mathbf{v}_{\mathbf{xy}}$ denote the target and actual base linear velocities, respectively ($w_{lin} = 1.5, \sigma_{lin} = 0.25$).
- $R_{ang} = \exp(-(\omega^*_z - \omega_z)^2/\sigma^2_{ang})$ is the angular velocity tracking reward, where ω^*_z and ω_z denote the target and actual yaw rates, respectively ($w_{ang} = 0.75, \sigma_{ang} = 0.25$).
- R_{air} denotes the foot clearance air-time reward to promote a distinct, symmetric gait ($w_{air} = 0.25$).
- P_i constitutes the penalty terms enforcing physical constraints: minimizing base vertical linear velocity, roll/pitch oscillations, deviations from nominal joint limits, excessive torque commands, rapid joint accelerations, and unintended structural contact ($w_{p,i}$ are the corresponding penalty weights).

3.3 3DGS Environment Construction & Agent Deployment

To construct a high-fidelity virtual verification sandbox, we reconstructed a campus hallway utilizing Structure-from-Motion (SfM), 3D Gaussian Splatting (3DGS), and FVDB. A sequence of high-resolution images of the physical corridor was captured using a DSLR camera and processed via COLMAP-based SfM to resolve camera poses. The reconstructed 3DGS scene was optimized and integrated with FVDB to instantiate a photorealistic, spatially consistent simulation asset. To support physical interaction, the scene geometry was compiled into a collision mesh and imported into Isaac Sim (v5.1.0) as a Universal Scene Description (USD) file. Static collision shapes were subsequently mapped to the floor and wall structures. The pre-trained DRL locomotion policy was then deployed directly on the Go2 model inside the reconstructed environment without further tuning. Despite the intricate geometric features and narrow corridor boundaries, the policy maintained robust gait characteristics, validating the effectiveness of the domain-randomized training regimen.

4. AUTONOMOUS NAVIGATION

4.1 Navigation System Configuration

The proposed navigation system consists of a cost-effective, LiDAR-free visual navigation pipeline incorporating an Intel RealSense D435i RGB-D sensor, the RTAB-Map visual SLAM framework, and the ROS 2 Nav2 stack. The D435i is rigidly mounted to the front torso of the Unitree Go2. Within the virtual sandbox, a simulated RGB-D sensor replicates matching color and depth profiles.

The navigation workflow is driven entirely by the synchronized streams. Initially, RTAB-Map processes the RGB stream to extract visual keypoint features (e.g., GFTT/ORB) for frame-to-frame Visual Odometry (VO), loop-closure detection, and pose-graph optimization. The aligned depth stream is then projected into 3D coordinates using these feature coordinates, constructing a local 3D octomap that is subsequently projected onto a 2D occupancy grid map for global path planning. Concurrently, to achieve low-latency local obstacle avoidance without a costly LiDAR instrument, raw depth frames are converted into a pseudo-2D laser scan using the `depthimage_to_laserscan` ROS 2 package. This synthesized scan is published as a `/scan` topic to populate the Nav2 local costmap. The Nav2 global planner generates optimal paths on the occupancy grid, while the DWB (Dynamic Window Approach B) local planner utilizes the local costmap to compute steering commands $\mathbf{v}_{\mathbf{cmd}}$ that safely route the robot around obstacles. Finally, these commands are transmitted to the active locomotion controller.

4.2 Validation in 3DGS Environment

Prior to real-world deployment, the integrated navigation stack was validated in the 3DGS-based virtual corridor (spanning $\sim$200 m in length and 2.5 m in width, featuring both straight and curved segments).

During simulation, the policy network receives proprioceptive and exteroceptive observations at each time step. Proprioception comprises joint positions and velocities, base angular velocity, and gravity vector projection. Exteroception is emulated using a virtual height-scan grid (1.6 m$\times$1.0 m at a resolution of 0.1 m) centered on the robot base. Ray-casting queries the reconstructed 3DGS collision mesh to feed local elevation profiles to the policy, enabling adaptive foot placement. The reference command vector $[v_x, v_y, \omega_z]^T$ is supplied directly by the DWB local planner.

Fig. 2. Visual SLAM and path planning validation inside the reconstructed 3DGS corridor environment. The reconstructed mesh provides accurate collision responses and realistic visual feedback.

To evaluate obstacle avoidance, virtual box obstacles were strategically placed along the corridor. As shown in Fig. 2, the local planner dynamically recalculated trajectories around the obstacles, and the RL locomotion policy adjusted its walking pattern to negotiate the detours without collision.

4.3 Physical Deployment and Results

Following virtual verification, the visual navigation pipeline was deployed on a physical Unitree Go2 Edu Plus quadrupedal platform. To adhere to the strict computational resource constraints of the edge computer, the PyTorch-based DRL locomotion policy was bypassed during physical deployment. Instead, the local trajectory commands ($\mathbf{v_{cmd}}$) generated by Nav2 were mapped directly to the robot's proprietary low-level motor controller via the Unitree Sport API. A lightweight ROS 2 bridge was developed to interface these components. This design significantly reduced the computational overhead on the NVIDIA Jetson Orin NX (operating in 20W power mode, equipped with 16GB RAM), reserving processing resources exclusively for RTAB-Map visual SLAM and Nav2 planning. The Intel RealSense D435i depth camera was directly connected to the Jetson Orin NX via USB 3.0.

A major challenge encountered during deployment was the operating system (OS) and middleware mismatch between the primary computing unit and the robot's built-in expansion dock. The Unitree Go2 Edu Plus dock is locked to Ubuntu 20.04 and ROS 2 Foxy due to proprietary firmware constraints. Conversely, the virtual simulation environment and the high-level autonomy stacks (RTAB-Map and Nav2) were developed on Ubuntu 24.04 and ROS 2 Jazzy. To bridge this version disparity without introducing intermediate translation layers or modifying the source code, we leveraged the native compatibility of the underlying Data Distribution Service (DDS) middleware. By configuring both environments to use CycloneDDS, standard message types—such as velocity command topics (`geometry_msgs/Twist`)—were seamlessly shared over the local network. This middleware alignment permitted direct, zero-code deployment of the navigation parameters tuned in the simulation environment.

Fig. 3. Real-world experimental setup. The Unitree Go2 robot is equipped with an Intel RealSense D435i and Jetson Orin NX, running the entire navigation stack on-board.

As shown in Fig. 3, RTAB-Map successfully executed loop closure detection in the physical environment, compensating for the high-frequency camera oscillations. Nav2 maintained stable global paths, allowing the robot to traverse the corridor at a target velocity of 0.4 m/s. The physical results verified that key visual SLAM and costmap parameters optimized under the simulated RL ego-motion noise (e.g., costmap inflation radius, voxel grid resolution, and visual loop-closure matching thresholds) were highly transferable to the physical platform, demonstrating the utility of our simulated testing environment.

5. CONCLUSION

This paper presented a practical, dual-stage visual navigation framework for quadrupedal robots. By leveraging a high-fidelity 3D Gaussian Splatting digital twin and simulating quadrupedal locomotion noise via an RL locomotion policy, we established a safe, photorealistic virtual validation sandbox for visual SLAM and path planning. For physical deployment, we bypassed the RL policy and routed commands directly to the robot's built-in Sport API via a custom ROS 2 bridge, optimizing edge computation on the Jetson Orin NX. Real-world experiments on the Unitree Go2 demonstrated that the parameters pre-tuned under simulated bobbing noise transferred successfully without manual in-situ recalibration, achieving robust visual tracking and obstacle avoidance.

Future work will focus on incorporating dynamic obstacle avoidance strategies and optimizing the processing pipeline to support higher-resolution visual feedback on the edge computer.

ACKNOWLEDGEMENT

This research was supported by the National Research Foundation of Korea (NRF) grant funded by the Korean government (MSIT) (No. RS-2025-24683458).

REFERENCES

[1] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile

and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, 2019.

[2] M. Labbé and F. Pomerleau, "Online global loop closure detection for large-scale multi-session graph-based SLAM," *Autonomous Robots*, vol. 34, no. 3, pp. 183–200, 2013.

[3] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022.

[4] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics (TOG)*, vol. 42, no. 4, pp. 1–14, 2023.